Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas de Información
y Ciencias de la Computación

ER-AFD / DOCUMENTO 03 DE 03

Manual de usuario

Convertidor de expresiones regulares a AFD

Construcción explícita ER → AFND → AFD

PRESENTADO POR

SELVIN KLEBER GIRÓN ALVAREZ

Carné: 3190-15-15973

Guatemala, 20 de octubre de 2026

1. Requisitos y preparación

ER-AFD es un programa de consola para convertir una expresión regular en AFND y AFD y evaluar una cadena. Una sola ejecución muestra ambas tablas y el resultado.

Requisito	Detalle
Entorno de entrega	Linux de 64 bits con una terminal compatible con UTF-8.
Python	Versión 3.10 o superior.
Make	Necesario para usar los objetivos del Makefile.
Bibliotecas	El programa y sus pruebas usan únicamente la biblioteca estándar.
Permisos	Usuario regular; el programa no necesita permisos de administrador.

Archivos que deben permanecer juntos

```
ProyectoFinalAutomatas/
  expresiones.py
  automatas.py
  er_afd.py
  er-afd
  Makefile
  pruebas.py
  doc/
    reporte_tecnico.pdf
    casos_de_prueba.pdf
    manual_de_usuario.pdf
```

Obtener la carpeta completa del proyecto y abrir una terminal dentro de ella. Los archivos de Python se importan entre sí, por lo que no deben moverse de forma aislada. No es necesario ejecutar pip install ni descargar bibliotecas adicionales.

El enlace público del repositorio aún no forma parte de esta entrega local. Cuando se publique, debe conservarse la misma estructura de archivos.

2. Ejecución en Linux

Preparar el programa

```
cd ProyectoFinalAutomatas
make config
make build
```

make config comprueba que Python sea al menos 3.10. **make build** también hace esa comprobación y otorga permiso de ejecución a er-afd. Al tratarse de Python, no genera un binario compilado.

Ejecutar la conversión y evaluación

```
./er-afd '(a|b)*abb' 'aabb'
echo $?
```

El primer argumento es la expresión regular y el segundo es la cadena. El ejemplo debe terminar con Cadena aceptada; el valor mostrado por echo será 1.

También puede utilizarse el objetivo run:

```
make run EXPRESION='(a|b)*abb' CADENA='aabb'
```

make run acepta los códigos 0 y 1 como ejecuciones válidas de la aplicación. Si se necesita consultar el código original de aceptación o rechazo, debe usarse el lanzador directamente. make comunica los errores de sus recetas con su propio código.

Limpiar y elegir intérprete

```
make clean
make config PITON=python3
```

clean elimina la carpeta __pycache__; conserva los programas y la documentación. La variable PITON permite elegir otro nombre de intérprete disponible.

Los comandos anteriores están preparados para Linux. Las capturas incluidas se obtuvieron en Windows; la validación de Make en Linux queda pendiente.

3. Argumentos, comillas y ejemplos

```
er-afd <ER> <cadena>
```

Usar comillas para proteger espacios y operadores del intérprete de comandos. En Linux conviene usar comillas simples para conservar literalmente las barras invertidas.

Necesidad	Comando de ejemplo en Linux
Aceptar una cadena	./er-afd '(a|b)*abb' 'aabb'
Rechazar una cadena	./er-afd '(a|b)*abb' 'abba'
Cadena vacía	./er-afd 'a*' ''
Espacios	./er-afd 'hola mundo' 'hola mundo'
Espacio por hexadecimal	./er-afd 'hola\h20mundo' 'hola mundo'
Asterisco literal	./er-afd '*' '*'
Entrada que empieza con guion	./er-afd -- '-a' '-a'
Consultar ayuda	./er-afd --help

Ejecución directa en Windows

Desde la carpeta del proyecto, ejecutar los archivos con Python. Esta es la forma utilizada para las evidencias; no se requiere Make para esta ruta de ejecución.

```
python er_afd.py "(a|b)*abb" "aabb"
python er_afd.py "[a-z][0-9]" "m5"
python er_afd.py --help
```

En PowerShell, el código del proceso puede consultarse con $LASTEXITCODE. El segundo argumento es texto literal: los escapes especiales los interpreta únicamente la expresión regular.

4. Expresiones admitidas

Símbolo o forma	Significado	Ejemplo
ab	Concatenación implícita.	ab acepta ab.
|	Unión o alternativa.	a|b acepta a o b.
*	Cero o más repeticiones.	a* acepta vacío y aaa.
(r)	Agrupación con mayor precedencia.	(a|b)c acepta ac y bc.
.	Exactamente un carácter ASCII.	. acepta 7; no acepta vacío.
ε	Cadena vacía.	aεb equivale a ab.
[abc]	Un carácter del conjunto.	[abc] acepta b.
[^abc]	Un ASCII fuera del conjunto.	[^abc] acepta z.
[a-z]	Rango inclusivo dentro del conjunto.	[a-z] acepta m.
\	Escapar un carácter literal.	* acepta *; \p acepta p.
\\	Barra invertida literal.	Acepta una sola barra invertida.
\N	Un dígito decimal.	\N acepta 5.
\hHH	Carácter ASCII por hexadecimal.	\h20 es espacio; \h41 es A.
\A	Una letra mayúscula.	\A acepta Z.
\a	Una letra minúscula.	\a acepta z.
\z	Una letra mayúscula o minúscula.	\z acepta A o a.

La coincidencia distingue mayúsculas de minúsculas. Los rangos deben ser ascendentes y sus extremos deben ser caracteres individuales. Los códigos hexadecimales válidos están entre 00 y 7F. Los signos + y ? se interpretan como caracteres literales.

5. Interpretación de la salida

Primero aparece el AFND y después el AFD. Q enumera los estados; Σ muestra el alfabeto de referencia; Inicial identifica q0 o Q0 y F contiene los estados finales. El AFND utiliza nombres con q minúscula y el AFD utiliza Q mayúscula.

```
AFND: q0|ε|{q1,q3}
AFD:  Q0|a|Q1
```

Cada fila contiene estado actual, símbolo y destino. El AFND puede mostrar un conjunto de destinos; el AFD muestra un único estado por clase. Si aparecen Clases de entrada, las filas utilizan esas clases disjuntas para resolver coincidencias entre el punto, literales y conjuntos negativos.



Ejemplo real: matriz del AFD y evaluación de aabb.

Código	Interpretación
1	Cadena aceptada.
0	Cadena rechazada.
2	Error en argumentos, expresión, codificación ASCII o anidamiento.

El código 1 representa aceptación por requisito del proyecto. Si un carácter no tiene transición, el recorrido muestra ∅ y la cadena se rechaza.

6. Errores y comprobación rápida

Mensaje o síntoma	Qué revisar
Falta cerrar el paréntesis	Completar el grupo; por ejemplo, cambiar (ab por (ab).
Falta cerrar el conjunto	Agregar el corchete de cierre.
El conjunto no puede estar vacío	Usar al menos un carácter dentro de los corchetes.
El rango debe estar en orden ascendente	Cambiar [z-a] por [a-z].
\h requiere dos dígitos hexadecimales	Escribir dos dígitos válidos; por ejemplo, \h41.
Solo caracteres ASCII	Revisar tildes, ñ y otros caracteres fuera de ASCII. ε solo es operador de la ER.
Demasiados paréntesis anidados	Reducir grupos innecesarios; la conversión se detiene con código 2.
Falta una expresión	Completar cada alternativa; para el vacío utilizar ε.

Repetir las pruebas automatizadas

```
python3 pruebas.py
# En Windows:
python pruebas.py
```

La salida correcta termina con OK. Si una comprobación falla, unittest informa el método y los datos del caso. La suite no necesita instalar bibliotecas adicionales.

Alcance de uso

Esta versión procesa una ER y una cadena por ejecución. No abre un menú gráfico, no recibe archivos fuente y no aplica múltiples reglas de tokenización. La explicación de los algoritmos está en reporte_tecnico.pdf y los resultados registrados están en casos_de_prueba.pdf y en las carpetas capturas/ y evidencias/.